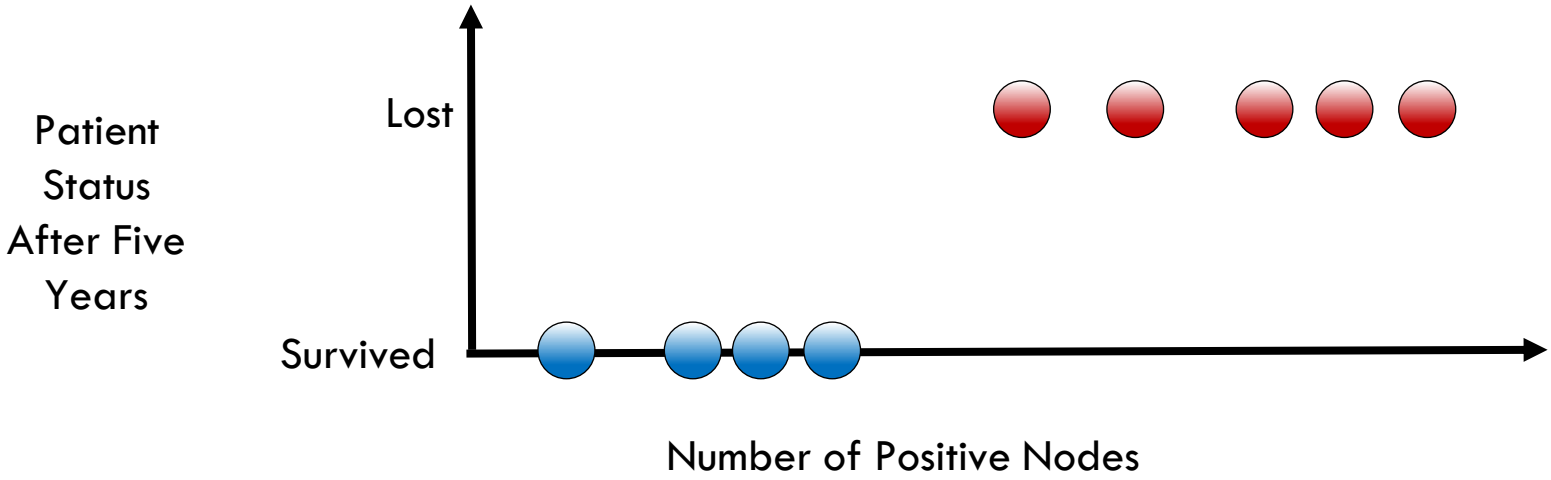
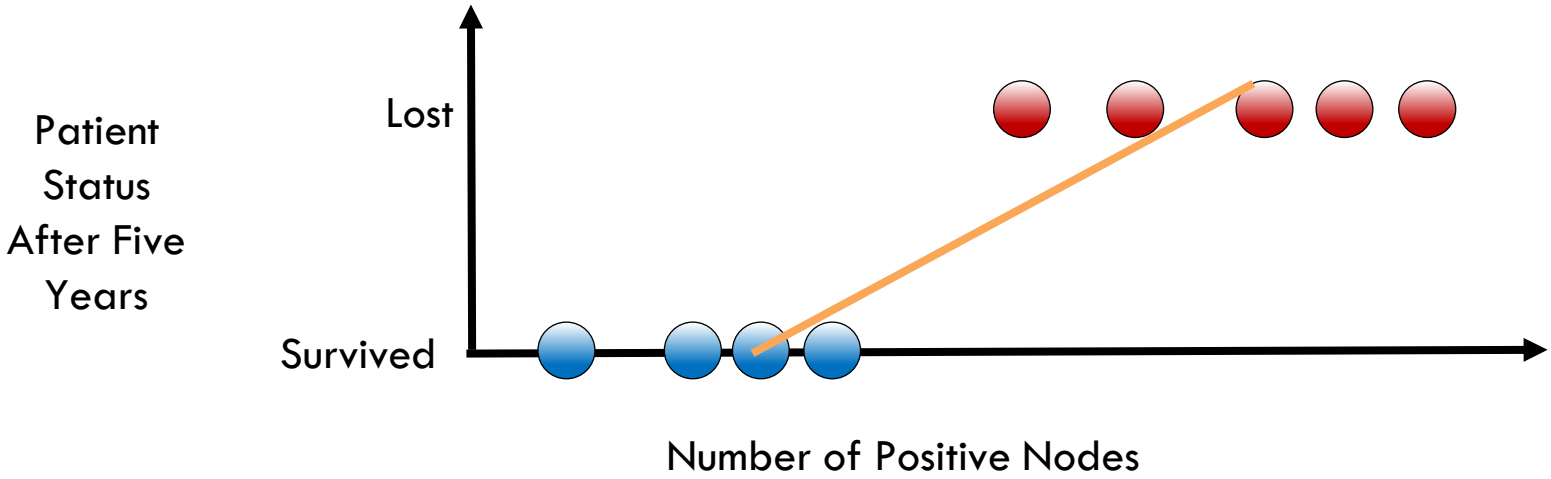
Logistic Regression

Introduction to Logistic Regression

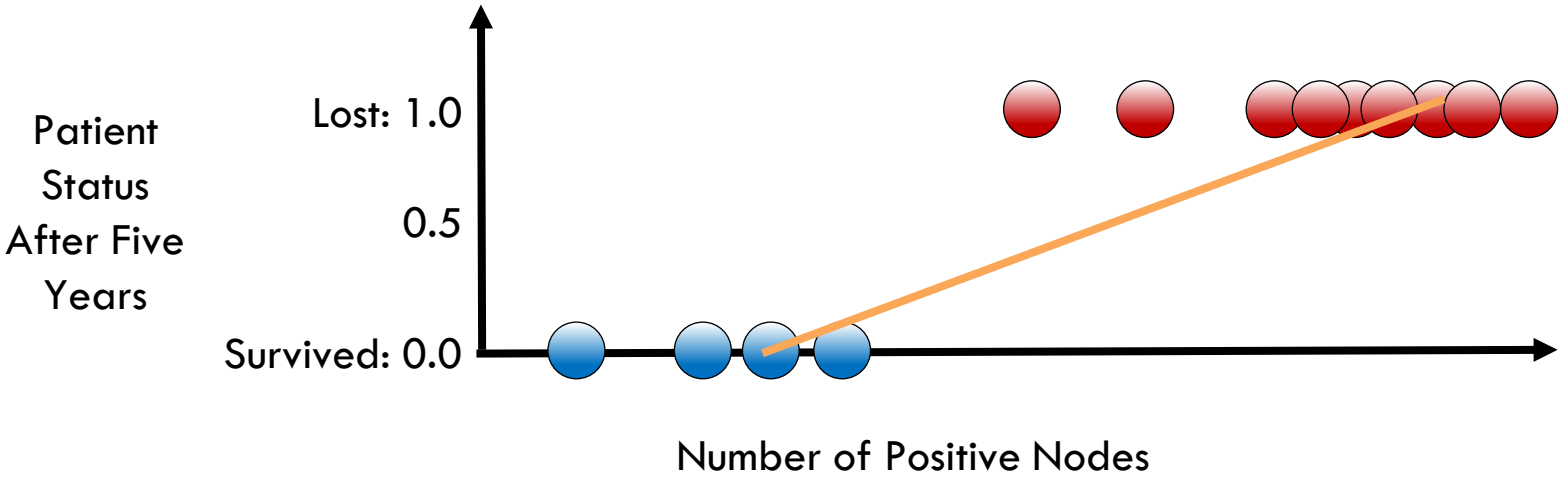


Linear Regression for Classification?



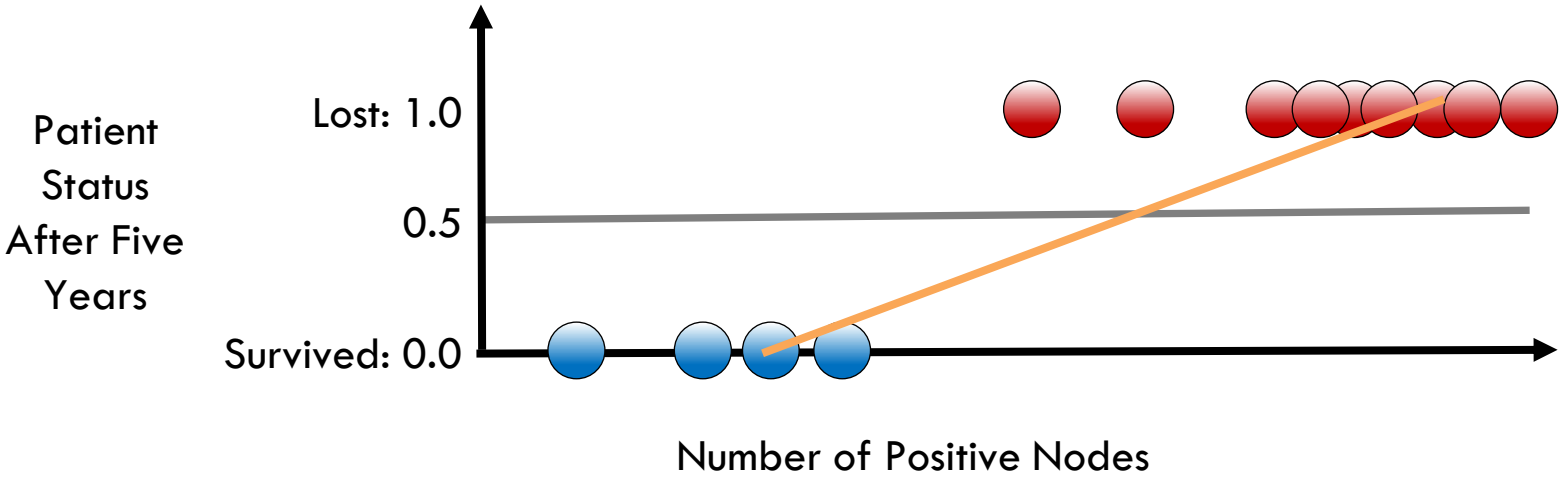
$$y_\beta(x) = \beta_0 + \beta_1 x + \varepsilon$$

Linear Regression for Classification?



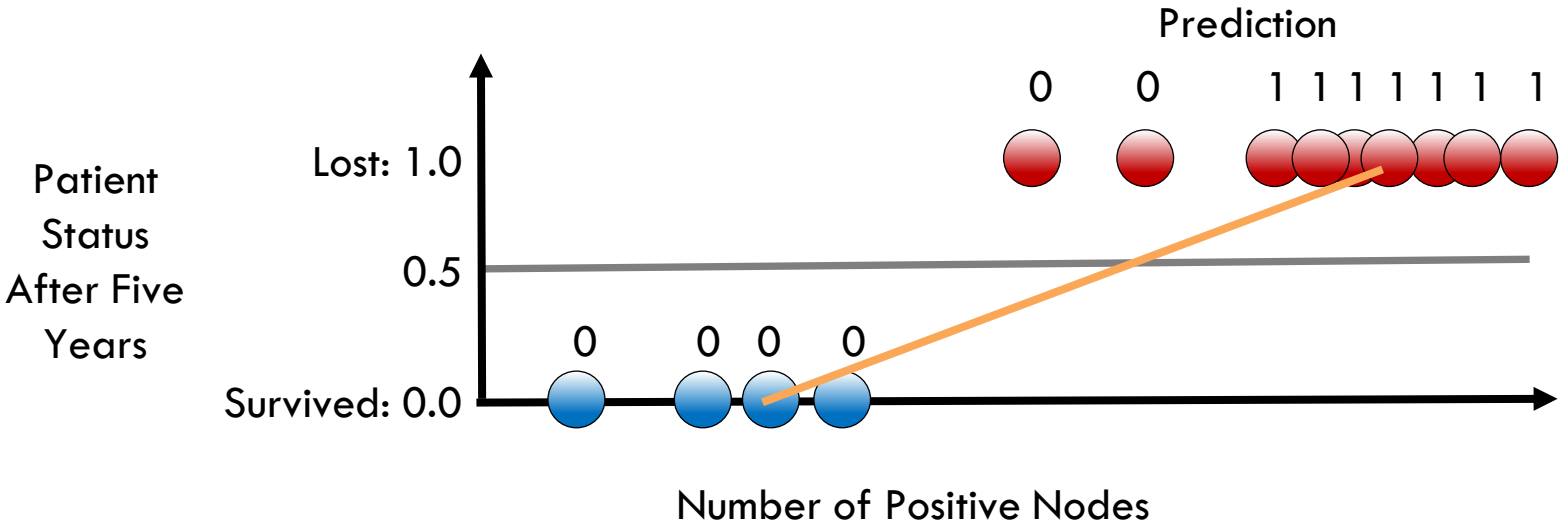
$$y_\beta(x) = \beta_0 + \beta_1 x + \varepsilon$$

Linear Regression for Classification?



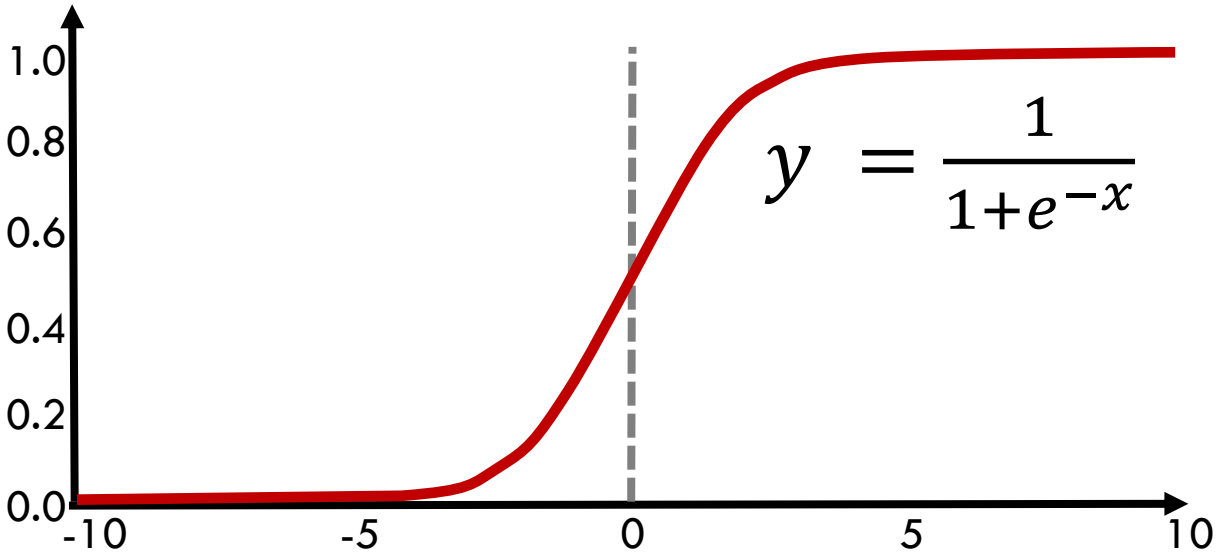
If model result > 0.5: predict lost
If model result < 0.5: predict survived

Linear Regression for Classification?

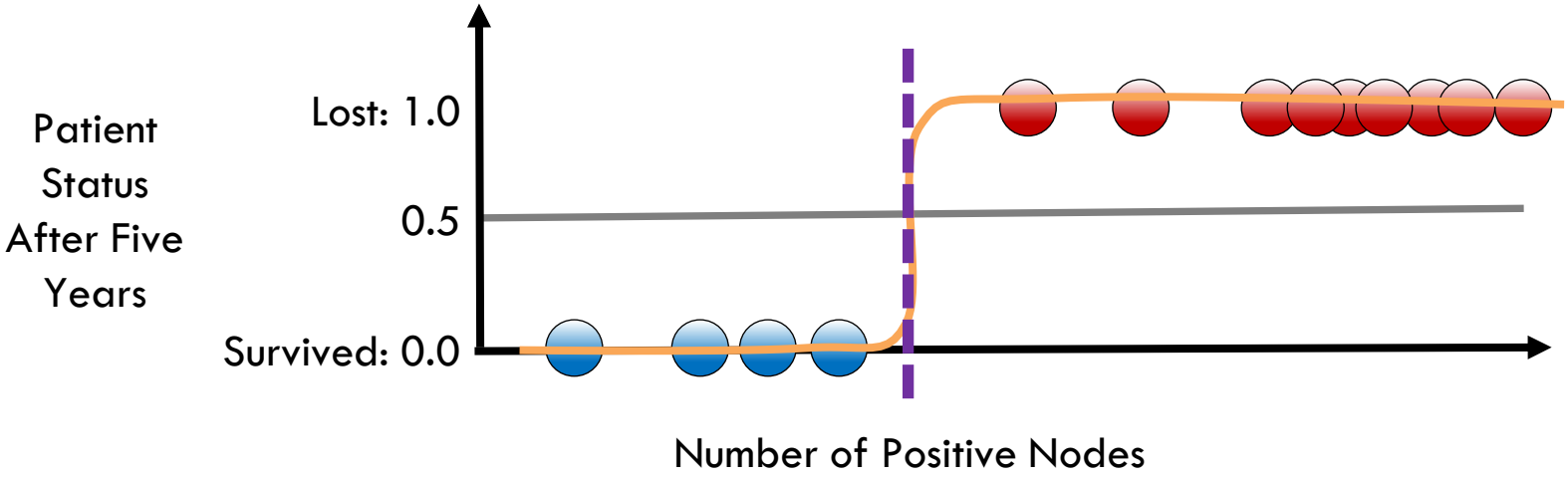


If model result > 0.5: predict lost
If model result < 0.5: predict survived

What is this Function?

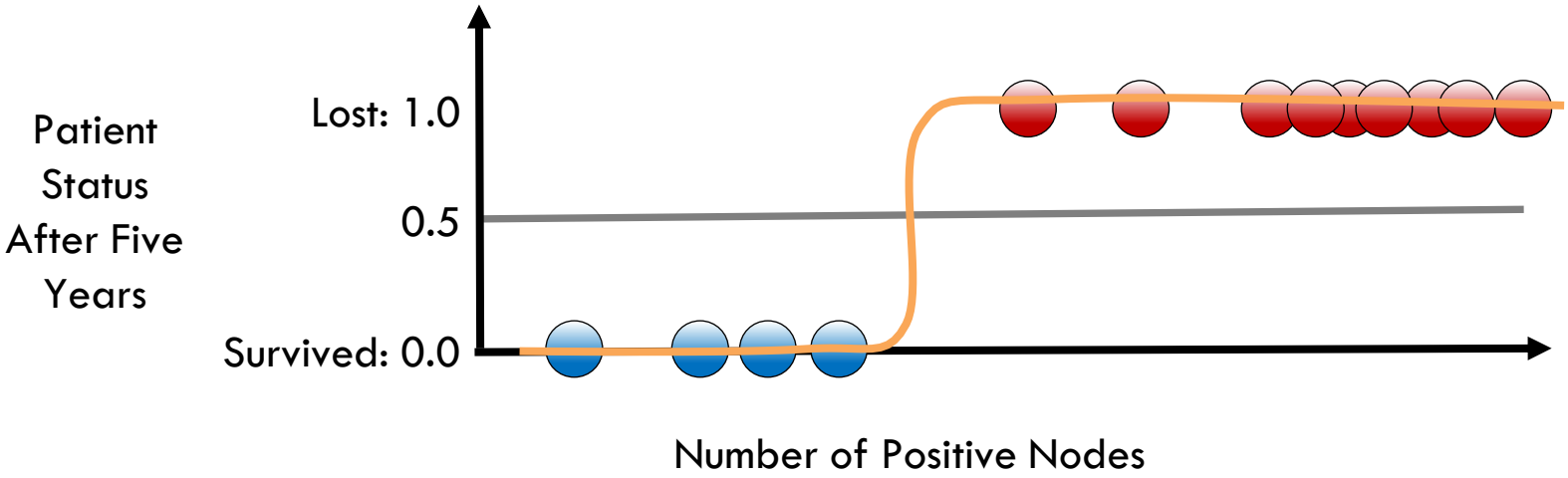


The Decision Boundary



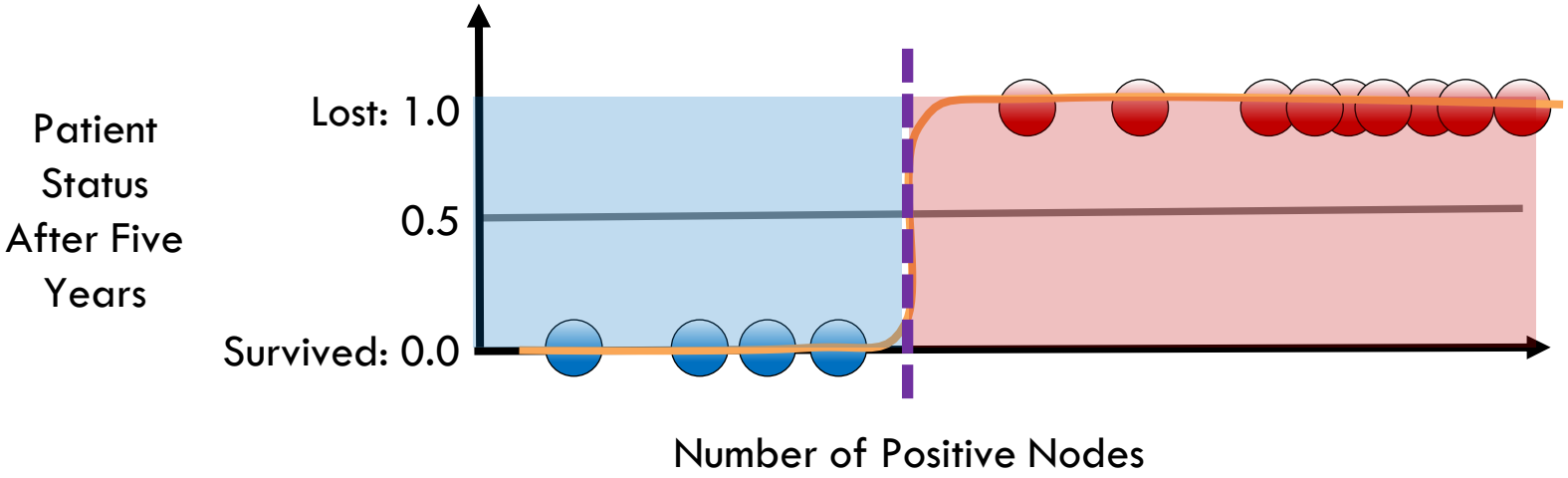
$$y_\beta(x) = \frac{1}{1+e^{-(\beta_0+\beta_1 x+\varepsilon)}}$$

Logistic Regression



$$y_\beta(x) = \frac{1}{1+e^{-(\beta_0+\beta_1 x+\varepsilon)}}$$

The Decision Boundary



$$y_\beta(x) = \frac{1}{1+e^{-(\beta_0+\beta_1 x+\varepsilon)}}$$

Relationship of Logistic to Linear Regression

Logistic Function

$$P(x) = \frac{1}{1 + e^{-(\beta_0 + \beta_1 x + \varepsilon)}}$$

Relationship of Logistic to Linear Regression

Logistic Function

$$P(x) = \frac{1}{1 + e^{-(\beta_0 + \beta_1 x + \varepsilon)}}$$

$$P(x) = \frac{e^{(\beta_0 + \beta_1 x)}}{1 + e^{(\beta_0 + \beta_1 x)}}$$

Relationship of Logistic to Linear Regression

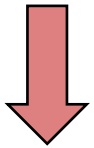
Logistic Function

$$P(x) = \frac{e^{(\beta_0 + \beta_1 x)}}{1 + e^{(\beta_0 + \beta_1 x)}}$$

Relationship of Logistic to Linear Regression

Logistic Function

$$P(x) = \frac{e^{(\beta_0 + \beta_1 x)}}{1 + e^{(\beta_0 + \beta_1 x)}}$$



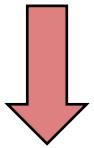
Odds Ratio

$$\frac{P(x)}{1 - P(x)} = e^{(\beta_0 + \beta_1 x)}$$

Relationship of Logistic to Linear Regression

Logistic Function

$$P(x) = \frac{e^{(\beta_0 + \beta_1 x)}}{1 + e^{(\beta_0 + \beta_1 x)}}$$



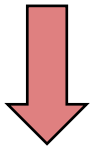
Log Odds

$$log\left[\frac{P(x)}{1 - P(x)}\right] = \beta_0 + \beta_1 x$$

Relationship of Logistic to Linear Regression

Logistic Function

$$P(x) = \frac{e^{(\beta_0 + \beta_1 x)}}{1 + e^{(\beta_0 + \beta_1 x)}}$$

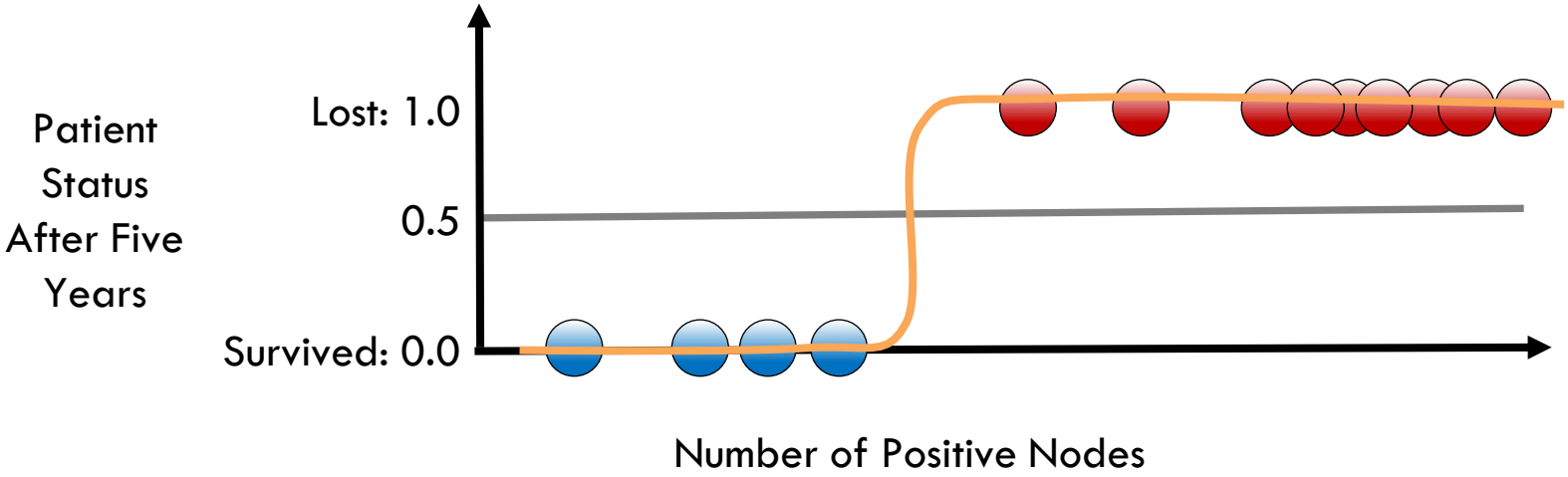


Log Odds

$$log\left[\frac{P(x)}{1 - P(x)}\right] = \beta_0 + \beta_1 x$$

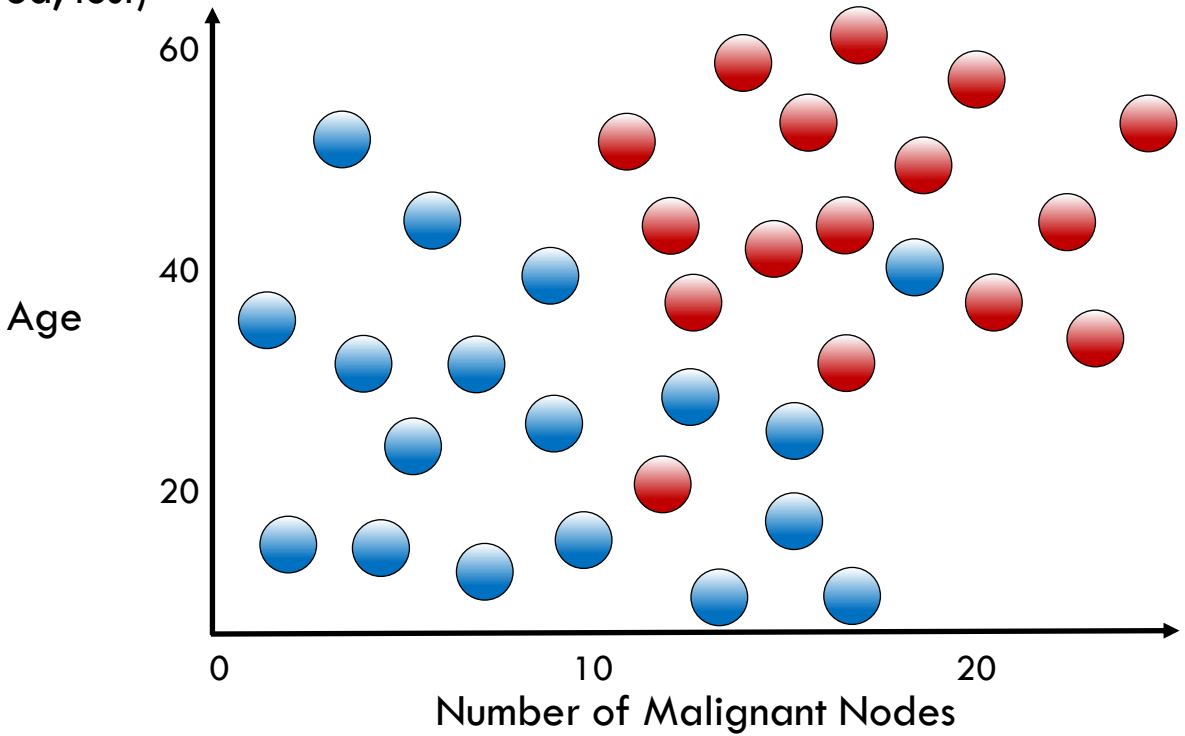
Classification with Logistic Regression

One feature (nodes)
Two labels (survived, lost)



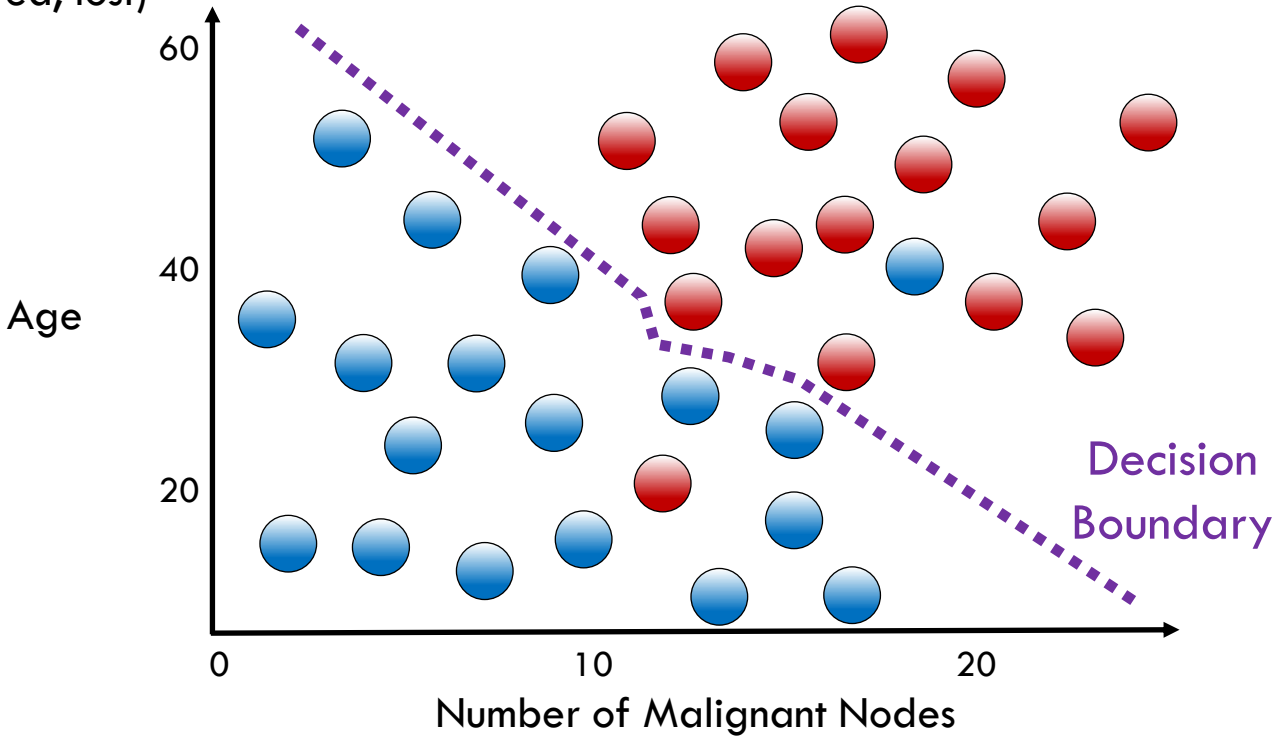
Classification with Logistic Regression

Two features (nodes, age)
Two labels (survived, lost)



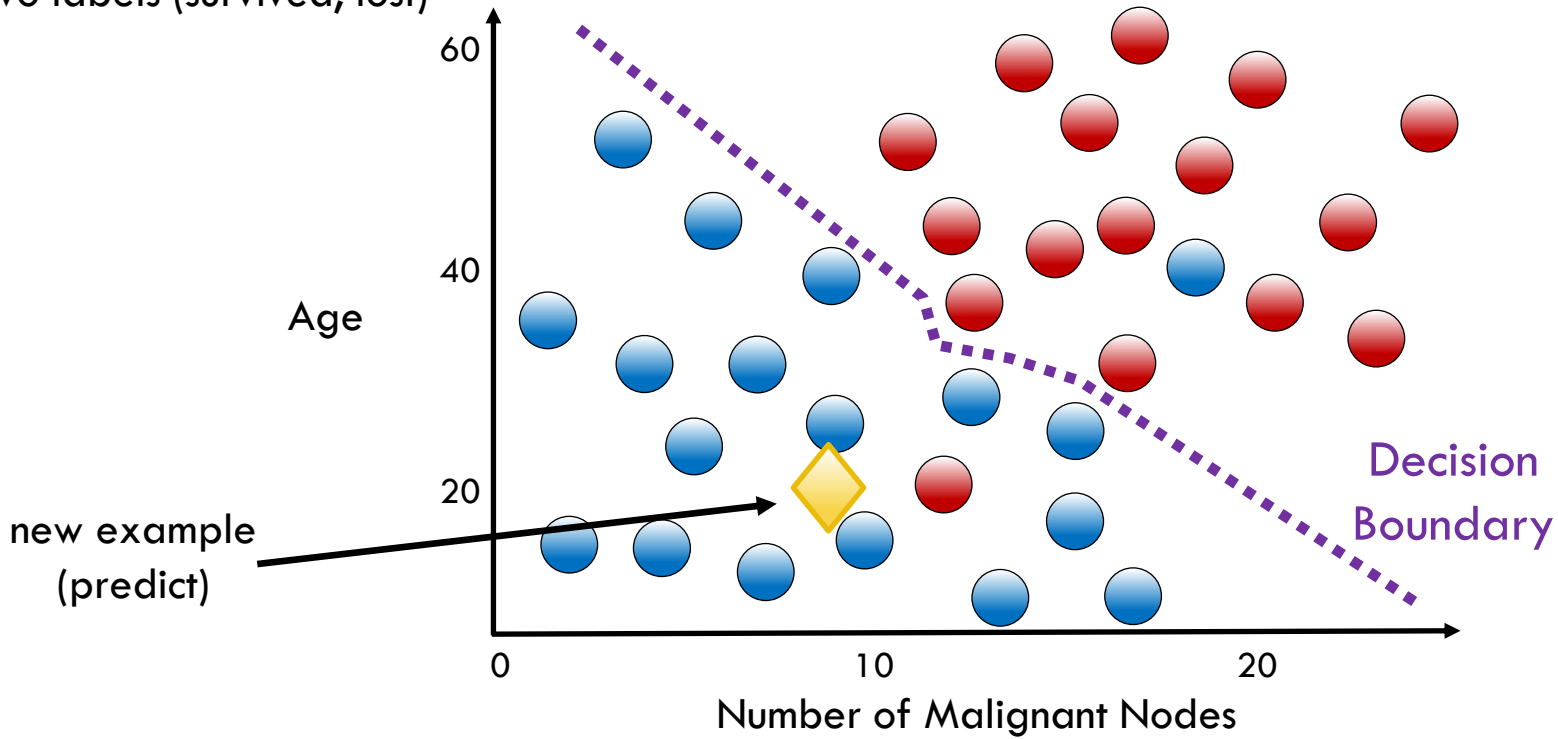
Classification with Logistic Regression

Two features (nodes, age)
Two labels (survived, lost)



Classification with Logistic Regression

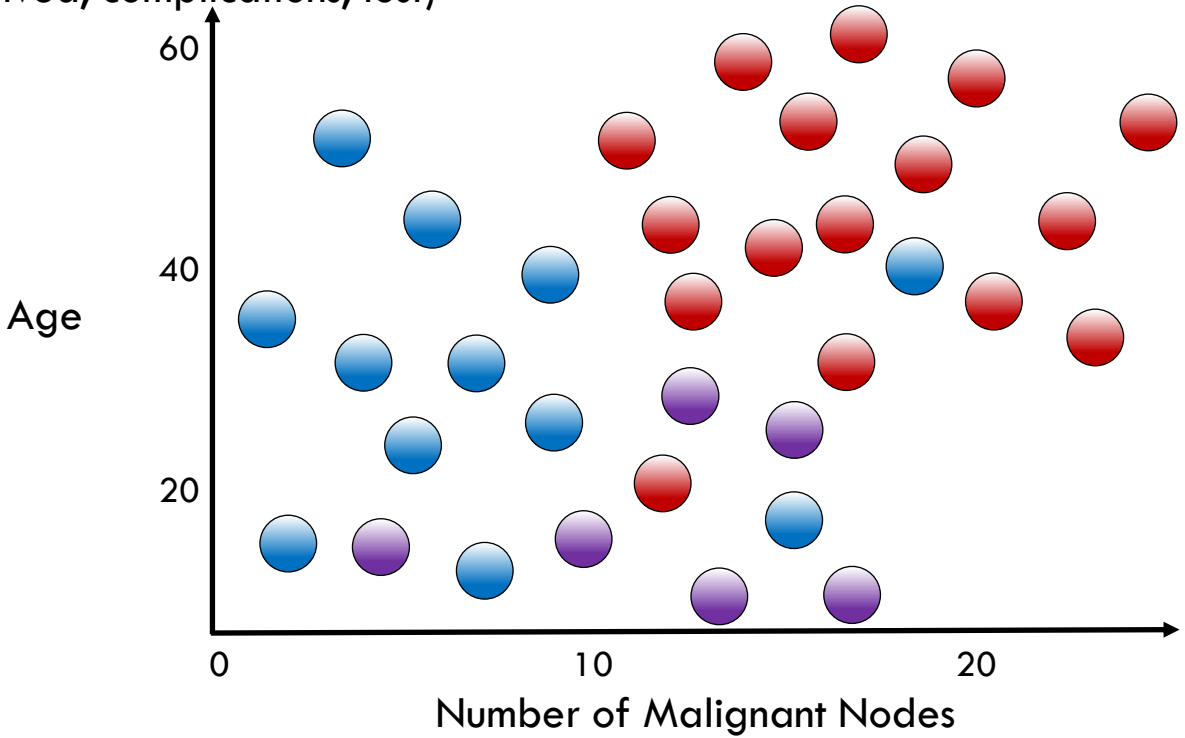
Two features (nodes, age)
Two labels (survived, lost)



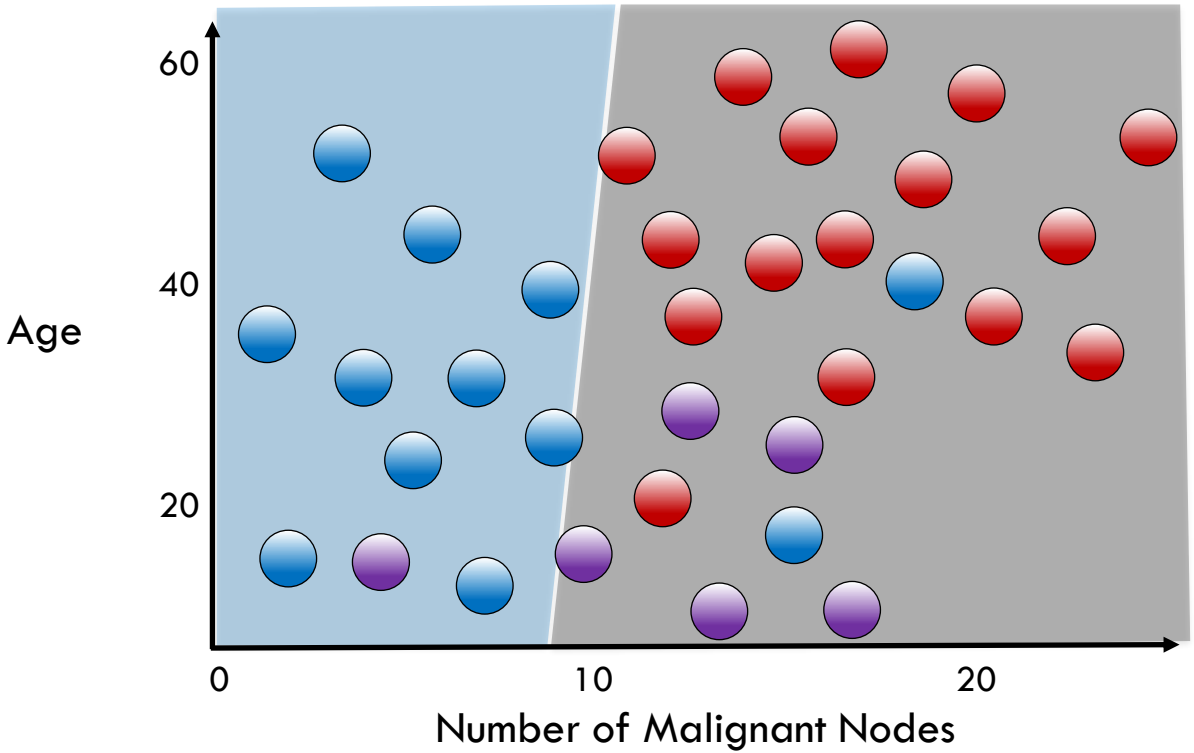
Multiclass Classification with Logistic Regression

Two features (nodes, age)

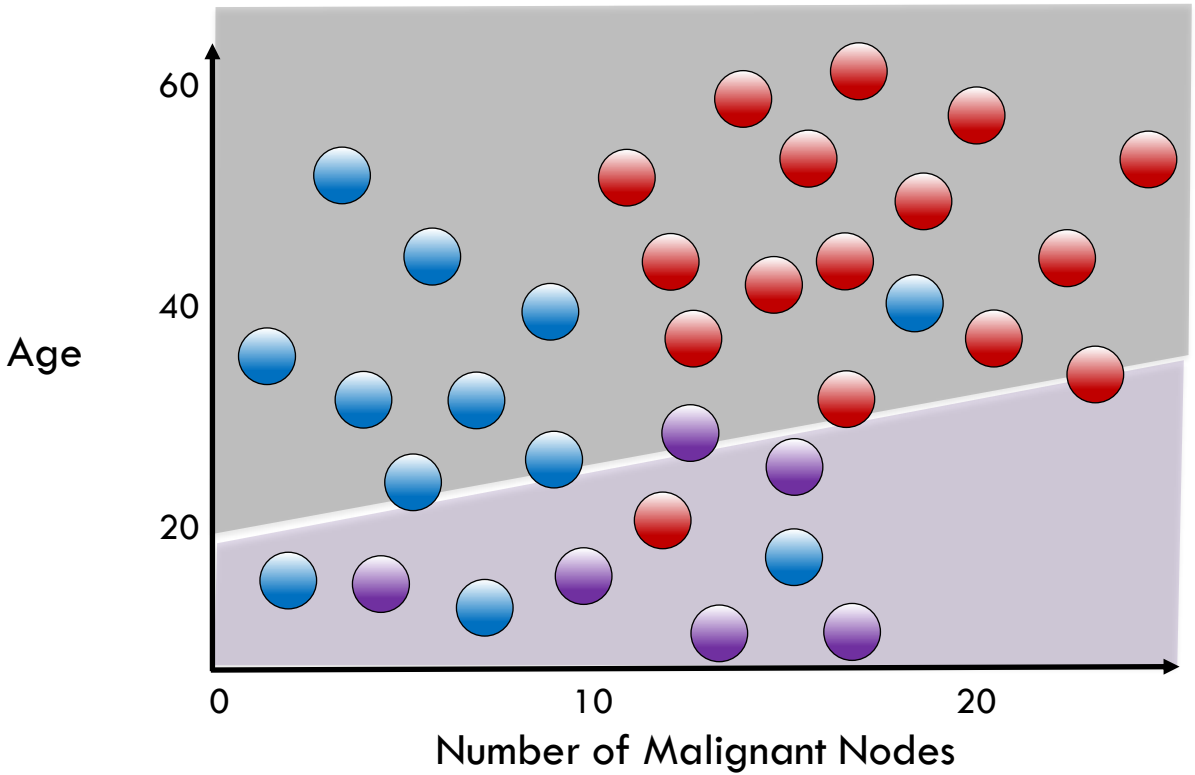
Three labels (survived, complications, lost)



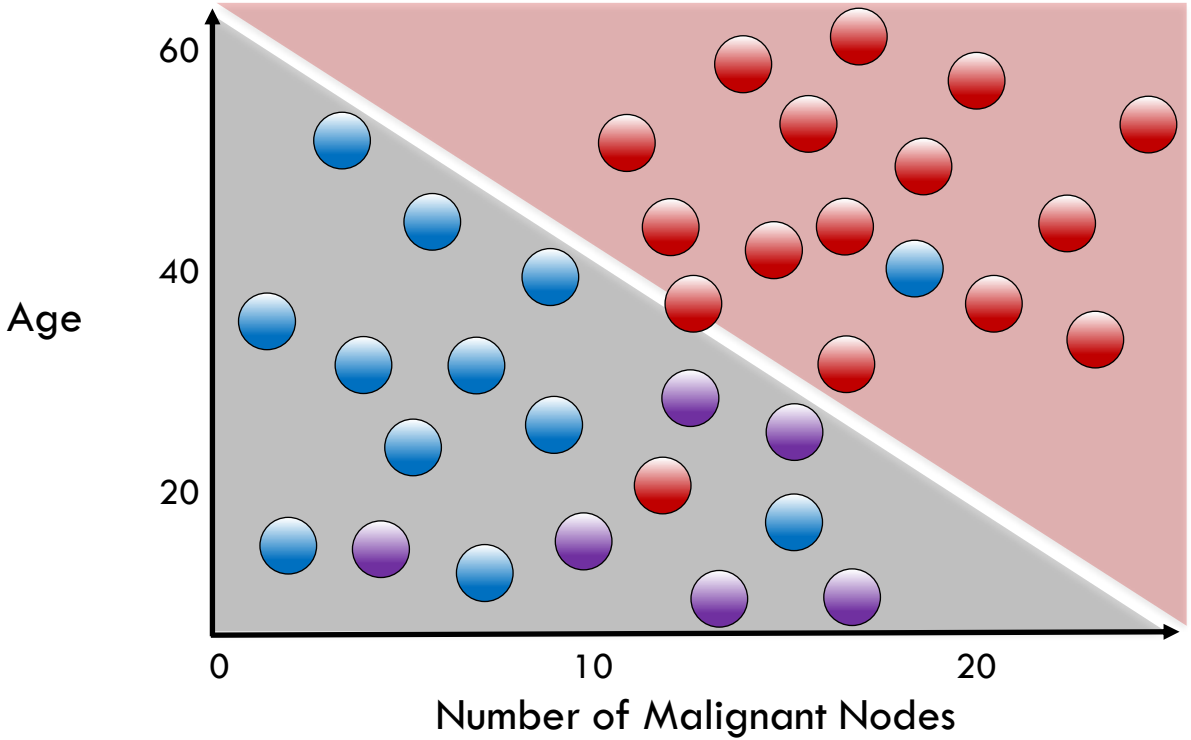
One vs All: Survived vs All



One vs All: Complications vs All

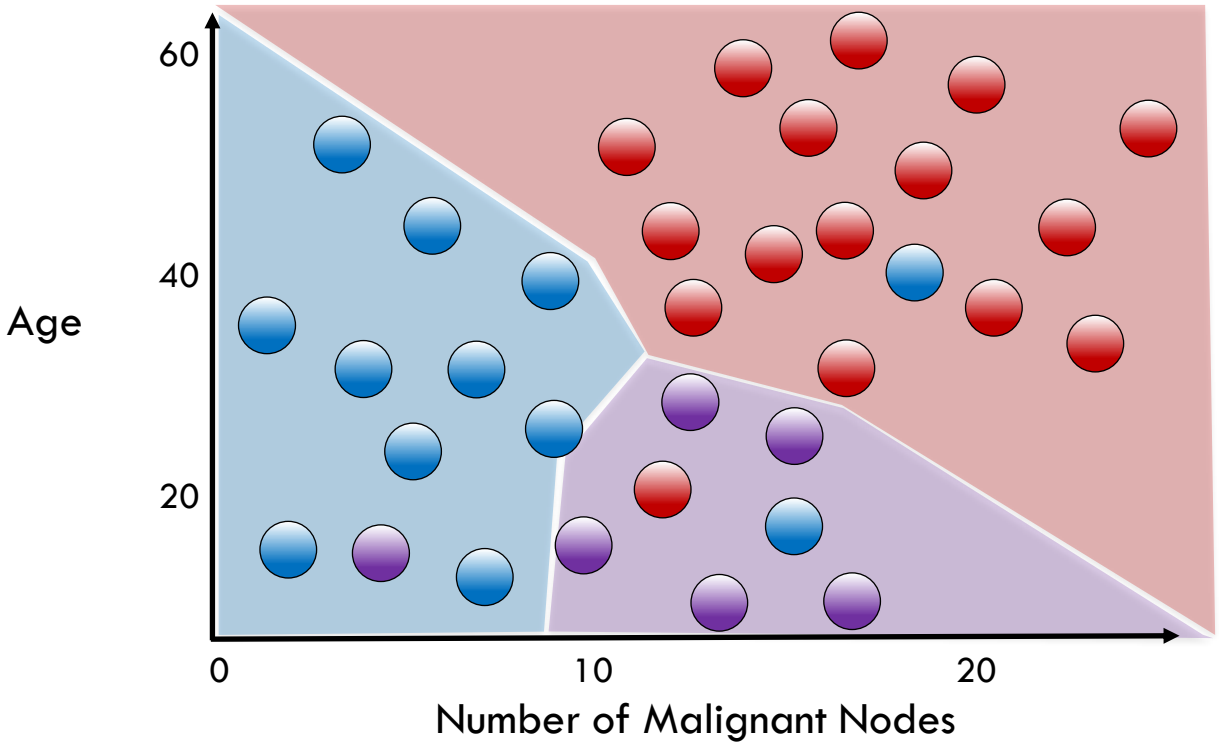


One vs All: Loss vs All



Multiclass Decision Boundary

Assign most probable class to each region



Logistic Regression: The Syntax

Import the class containing the classification method

```
from sklearn.linear_model import LogisticRegression
```

Logistic Regression: The Syntax

Import the class containing the classification method

```
from sklearn.linear_model import LogisticRegression
```

Create an instance of the class

```
LR = LogisticRegression(penalty='l2', c=10.0)
```

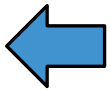
Logistic Regression: The Syntax

Import the class containing the classification method

```
from sklearn.linear_model import LogisticRegression
```

Create an instance of the class

```
LR = LogisticRegression(penalty='l2', c=10.0)
```



regularization parameters

Logistic Regression: The Syntax

Import the class containing the classification method

```
from sklearn.linear_model import LogisticRegression
```

Create an instance of the class

```
LR = LogisticRegression(penalty='l2', c=10.0)
```

Fit the instance on the data and then predict the expected value

```
LR = LR.fit(X_train, y_train)
y_predict = LR.predict(X_test)
```

Logistic Regression: The Syntax

Import the class containing the classification method

```
from sklearn.linear_model import LogisticRegression
```

Create an instance of the class

```
LR = LogisticRegression(penalty='l2', c=10.0)
```

Fit the instance on the data and then predict the expected value

```
LR = LR.fit(X_train, y_train)
y_predict = LR.predict(X_test)
```

Tune regularization parameters with cross-validation: LogisticRegressionCV.